

1. A comparison asks a question — and the answer is a value

```
health = 24

print(health < 30)      # True
print(type(health < 30)) # <class 'bool'>
```

Read it out loud: “is health less than 30?” Python answers. That answer is a `bool` — the fourth type from Unit 1. It is a real value, the same way `2 + 2` is a real value.

2. Give the answer a name

```
health = 24
xp = 1450
shields = 0

is_hurt = health < 30      # True
is_veteran = xp >= 1000    # True
has_shield = shields > 0   # False
```

Nothing new is happening — it is ordinary assignment. But storing the answer is what lets you read it, print it, and reuse it instead of asking the same question three times.

3. if means “when”

When health drops below 30, **warn** the player

if	WHEN — the question
<condition>	
:	then — ends the line, not optional
body	do this — indented 4 spaces

Say the rule in English first. Then write it.

The six comparison operators

OPERATOR	ASKS
==	Are these equal?
!=	Are these different?
<	Is the left smaller?
<=	Smaller or the same?
>	Is the left bigger?
>=	Bigger or the same?

There is no `===` in Python. `==` is the only equality test.

Name it like a yes/no question

PREFIX	MEANS
is_	a state — <code>is_hurt</code> , <code>is_banned</code>
has_	possession — <code>has_shield</code> , <code>has_pet</code>
can_	ability — <code>can_afford_skin</code> , <code>can_enter</code>

Why it matters. `hurt = health < 30` could be holding a number, for all a reader knows. `is_hurt` can only be holding an answer.

A **convention**, not a rule. Python will not stop you — just as it did not stop you writing `width` instead of `WIDTH` in Unit 1.

```
if is_hurt:
    print("Low health!")
    print("Find a medkit")

print("Match continues")
```

The two indented lines are inside the “when”. The last line is not, so it runs every time — set `health` to 90

and it is the only thing that prints.

The indentation IS the block. There is no closing marker. The block starts after the colon and ends when the indentation ends.

One equals sign assigns. Two compare. `health = 30` puts 30 into `health`. `health == 30` asks whether it already is 30. `if health = 30:` is a `SyntaxError` — Python stops you before the program runs.

4. `else` — the other path

```
health = 90

if health < 30:
    print("Low health!")
else:
    print("You're fine")
```

Exactly one branch runs. Never both. Never neither.

`else` takes a colon but no condition. `else health >= 30:` is a `SyntaxError`. The `else` catches everything the `if` did not.

Without the `else`, a player at 90 health is told *nothing at all*. Often that silence is the bug.

5. Decide, then use — the pattern for the whole unit

Do this

```
kills = 14

if kills >= 10:
    rank = "MVP"
else:
    rank = "Player"

print(f"Rank: {rank}") # once
```

Not this

```
if kills >= 10:
    print("Rank: MVP")
else:
    print("Rank: Player")
                                # written twice
```

The second version works. It is still worse: when the wording changes you must remember *both* places — and eventually you will not.

Branches decide a value. Code after them uses it. This shape returns every time you branch for the rest of the unit.

6. Words you should be able to define

TERM	MEANS
Boolean (<code>bool</code>)	The type with only two values, <code>True</code> and <code>False</code> . Both capitalized.
condition	An expression that evaluates to <code>True</code> or <code>False</code> .
comparison operator	An operator that compares two values and produces a <code>bool</code> .
header	The <code>if</code> line. Always ends with a colon.
body	The indented lines under the header. PEP 8: four spaces, never tabs.
branch	One path through an <code>if / else</code> .

7. Mistakes to expect

MISTAKE	WHAT GOES WRONG
<code>if health = 30:</code>	<code>SyntaxError</code> . Use <code>==</code> to compare.
<code>if health < 30</code>	Missing colon. <code>SyntaxError</code> pointing at the next line.
Body not indented	<code>IndentationError: expected an indented block</code> .
Tabs mixed with spaces	<code>TabError</code> . Looks fine on screen and still breaks. Use spaces.
<code>else health >= 30:</code>	<code>else</code> never takes a condition. Just <code>else: .</code>
<code>true / false</code>	<code>NameError</code> . Python capitalizes <code>True</code> and <code>False</code> .
Comparing <code>input()</code> without converting	Compares text, not numbers. <code>"9" > "17"</code> is <code>True</code> .
Printing inside both branches	Runs correctly, but the line is now written twice.

8. Before you submit

- Does every `if` and `else` line end with a colon?
- Is every body indented four spaces, spaces and not tabs?
- `==` to compare, `=` to assign?
- Did you convert with `int()` before comparing numbers?
- Are your booleans named `is_` / `has_` / `can_` ?
- Does the branch **decide** a value, with the print once, afterwards?
- **Run it twice** — once so the `if` fires, once so the `else` does. A branch you never ran is a branch you never checked.

What is next. You can now split a program into two paths. Try three: under 1000 XP is Member, 1000–4999 is Veteran, 5000+ is Legend — using only `if` and `else` . You will end up burying one `if` inside another, and it grows worse with every role added. Next lesson introduces `elif` , which handles as many branches as you need without the nesting.

Keep this sheet. Every program from here on decides something. The colon, the indentation, and the difference between `=` and `==` do not change — they just get used inside loops instead of on their own.